HW6: Scalability — Utkarsh Sharma · CS6650

Code and full evidence: github.com/utkarshsharma19/Building-Scalable-Systems-CS6650/tree/main/hw06

Setup. Gin + sync.Map stores 100,000 products; each search checks exactly 100 and returns at most 20. Two laptop Locust workers used FastHttpUser and 0–10 ms think time. ECS Fargate ran in Oregon; Grafana measured the app and CloudWatch measured AWS resources.

Part II: load, bottleneck and vertical scaling

CPU units / users	req/s	p95 (ms)	CPU	Failures
256 / 5	161	35	~25%	0
256 / 20	520	43	~62%	0
256 / 60	1,056	90	~99%	0
512 / 60	1,488	52	72–75%	0
512 / 120 (5 min)	2,108	100	95%*	0

Interpretation. At 256 units, throughput flattened near 1,050 req/s while CPU reached 99% and client p95 rose. Memory stayed near 10%; work remained 100 checks and handler p95 about 0.25 ms. That supports adding compute, though reducing per-request CPU could also help. The 512/60 run improved throughput 41% and p95 42%; 120 users used more of that headroom, with zero failures in 632,473 requests.

CloudWatch decision. For a 256-unit task, ~1,000 req/s is the measured saturated capacity; ~700 req/s/task is an initial 70%-utilization target, to validate under real traffic. At 120 users, 512 units delivered ~2,108 req/s: twice the 256-unit ceiling. *The first full-load minute averaged 95.2% CPU (max 98.9%); lab access was revoked before later points could be retrieved. The higher p95 (100 ms) suggests queueing near CPU saturation; 60 users had not exposed a hard network limit.

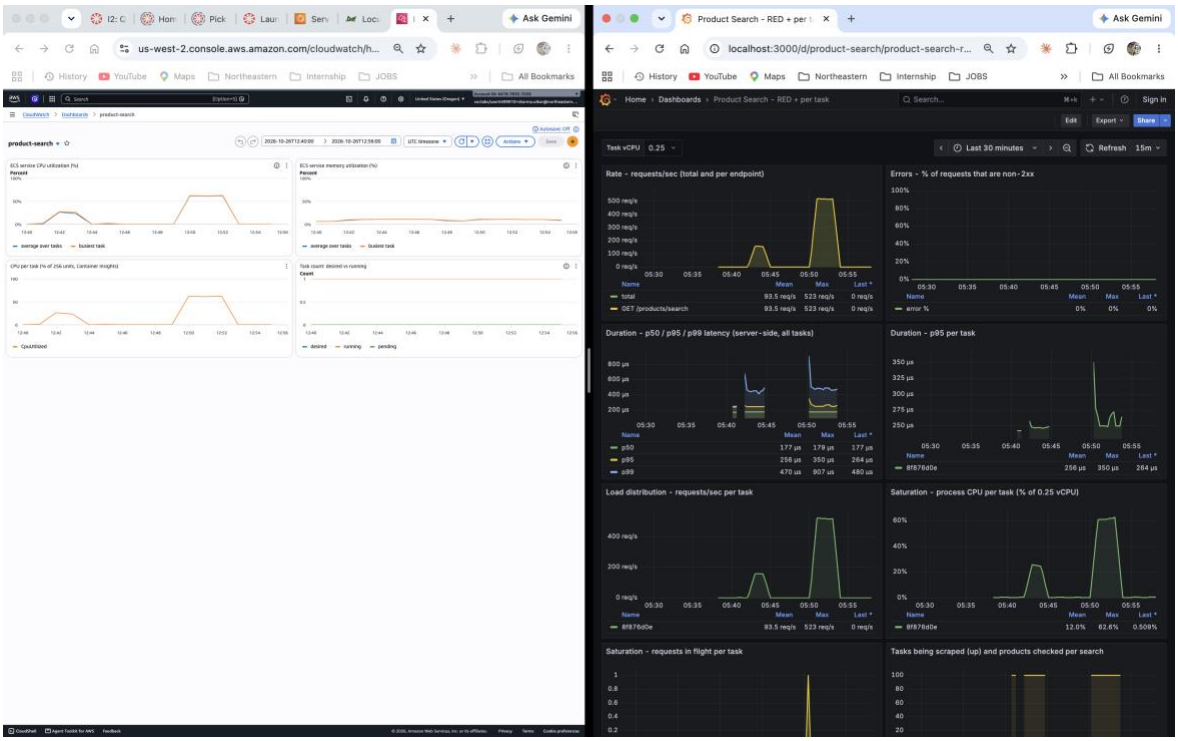


Fig. 1. Same Part II tests: CloudWatch left (UTC 12:40–12:56), HW5 Grafana right (PDT 05:40–05:56). Full-resolution screenshots are in the repository.

What each adds. CloudWatch exposes CPU/memory saturation and task counts; Grafana adds request rate, errors, per-task distribution and checks/search. App handler timing misses waiting before the handler: low server p95 did not mean low client latency. Use resource CPU together with client/ALB latency; Grafana updates every 5 s, while CloudWatch has one-minute points and ingestion delay.

Part III: horizontal scaling and stress exploration

Components and result. The ALB is the public entry point; its target group tracks task IPs and /health checks. Application Auto Scaling adjusts ECS desired count from CloudWatch CPU alarms (70% target, 2–4 tasks, 300 s cooldowns); ECS starts tasks and registers targets. Distributing the CPU work removed the single-task bottleneck: the 60-user core test delivered ~1,450 req/s, p95 60 ms and 0 failures over 1.68 million requests, versus 1,056 req/s and p95 90 ms in Part II.

Core scale-out evidence. Load began 06:16:40 PDT. The timeline first recorded desired count 3 at 06:23:00 and three healthy targets at 06:23:33 (~7 minutes to capacity). CloudWatch showed average CPU ~74%, then below 70% after scale-out. Grafana showed ~717/716 req/s across two tasks, then ~470/task across three; handler p95 stayed sub-millisecond and in-flight requests mostly 0–1. Stopping one task at 06:25:32 caused no Locust failures; replacement took about a minute.

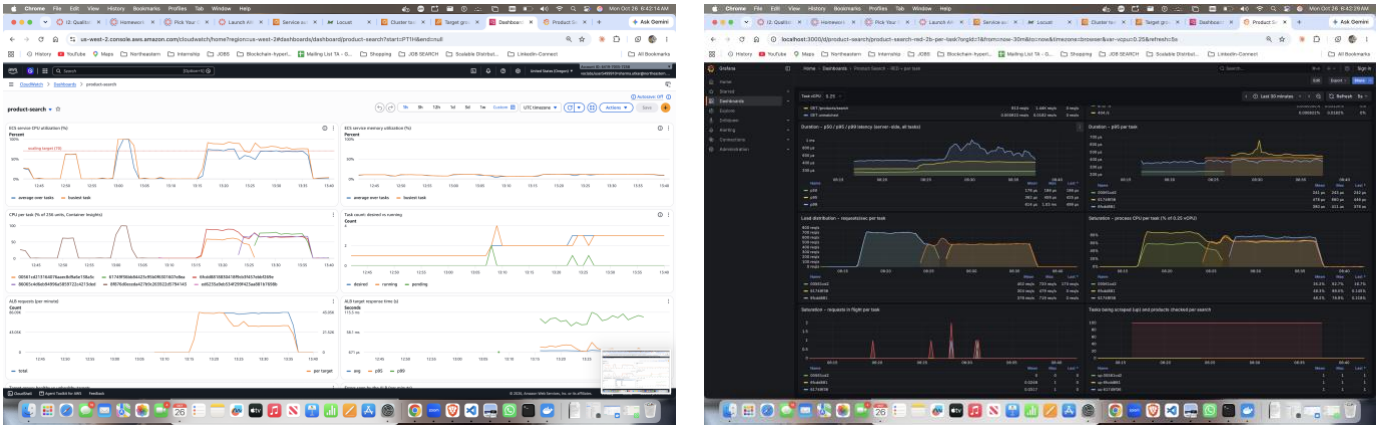


Fig. 2. Core run: CloudWatch CPU/task count/ALB requests (left, UTC) and Grafana latency, per-task rate/CPU and in-flight requests (right, PDT). UTC = PDT + 7 hours.

CPU vs request metrics. For this fixed-work search, I would test ALBRequestCountPerTarget at ~42,000 requests/target/minute (700 req/s). The core dashboards show traffic was already high well before extra capacity arrived, while app latency/in-flight signals gave little warning. Request count per target directly measures offered work; CPU remains a useful guardrail. This policy was not tested, and changing metrics alone does not remove CloudWatch evaluation or task-start delays. Use client/ALB latency to check service quality, not as the sole scaling trigger.

Horizontal vs vertical. A bigger task is simpler but has a size limit and is still a single point of failure. More tasks scale further and survive failures, at the cost of an ALB, slower reaction (~7 min) and no shared state between tasks.

Creative stress tests and predictions. Spike 10→80→10 users: CPU 90–97%; the extra task became ready ~40 s after the high-load phase ended, with no failures—short spikes need spare or scheduled capacity. Ramp +10 users/2 min, 50% target and 60 s cooldowns: scaled 2→3→4, ~2,000 req/s and p95 61 ms; gradual loads fit reactive scaling better, at higher cost. A 90% target would run hotter (prediction, not tested). Stopping 3/4 tasks at 80 users recovered in 67 s with 0 failures and a latency bump. A broken health-check path made all targets unhealthy: ALB failed open, but ECS repeatedly replaced tasks until the path was fixed; search failures remained zero. The 120-user vertical follow-up adds a concurrency stress test.

Supporting evidence: results/HW6-detailed-evidence.docx, scaling timeline CSV, raw Locust/CloudWatch results and screenshots/. Initial AWS teardown completed; follow-up teardown is blocked by revoked lab credentials until access is restored.